UNIVERSIDAD AUTÓNOMA DE CIUDAD JUÁREZ

Instituto de Ingeniería y Tecnología

Departamento de Ingeniería Eléctrica y Computación

Campus Ciudad Universitaria



Integración y entrega continua para sistemas legado en producción: contenedorización y entornos de prueba bajo demanda sobre infraestructura autoalojada

Alumno:

Oscar Alonso Nava Rivera 214729

Requisito para la obtención del título de

INGENIERO EN SOFTWARE

Director:
Mtro. Abraham López Najera

Ciudad Juárez, Chihuahua 21 de noviembre de 2026

Introducción

En la actualidad, la agilidad y confiabilidad en la entrega de software son fundamentales para garantizar la continuidad operativa de las organizaciones. Prácticas modernas como la Integración y Entrega Continuas (CI/CD) han demostrado reducir los riesgos de despliegue y mejorar el desempeño de los equipos técnicos. Sin embargo, aplicar estos principios a sistemas heredados (*legacy*) que operan sobre infraestructura autoalojada y que carecen de pruebas automatizadas representa un desafío arquitectónico y organizacional de alta complejidad.

El presente proyecto se centra en un sistema administrativo y operativo crítico desarrollado en Laravel, cuyo proceso de pase a producción es completamente manual. A pesar de contar con un equipo de desarrollo activo y un repositorio con más de 8,000 *commits*, el despliegue requiere intervención directa por SSH para ejecutar actualizaciones de código y migraciones coordinadas entre dos bases de datos distintas (MySQL y MongoDB). Esta dependencia manual, sumada a la ausencia de entornos de prueba efímeros, genera un alto riesgo de error humano por omisión, lo que se refleja en un *lead time* mediano de 55 minutos y una tasa de fallos post-despliegue del 21.9 %.

Para mitigar estos riesgos, es indispensable transitar hacia un proceso de entrega estandarizado y con trazabilidad. Implementar un *pipeline* de automatización eliminará la dependencia de las ejecuciones manuales en el servidor, mientras que la creación de entornos de prueba bajo demanda permitirá validar los cambios de forma segura antes de integrarlos, protegiendo así las operaciones críticas de la organización.

Este protocolo de investigación tiene como objetivo general diseñar e implementar un proceso de integración y entrega continua para dicho sistema, basado en contenedores y operado sobre infraestructura autoalojada, que reduzca los pasos manuales y permita la validación preventiva de los cambios.

Para lograrlo, se propone una estrategia de modernización progresiva. Mediante una metodología ágil iterativa, se empaquetará la aplicación web en contenedores Docker y se configurará un *pipeline* en GitLab CI/CD. Ante la falta de pruebas automatizadas dinámicas, la calidad se asegurará mediante compuertas de análisis estático y seguridad. Asimismo, se automatizará el despliegue de entornos de prueba efímeros con datos inyectados por *seeders*, culminando en un pase a producción seguro, coordinado y protegido por aprobación manual.

El resto del documento está estructurado de la siguiente manera. En primer lugar, se detalla el planteamiento del problema, los antecedentes y los objetivos, seguido del marco teórico que fundamenta los conceptos de CI/CD y métricas DORA. Posteriormente, se presenta a detalle la arquitectura del producto propuesto, la metodología de seis fases a seguir para su desarrollo técnico y el cronograma de actividades. Finalmente, se exponen los resultados esperados tras la culminación del proyecto.

1. Planteamiento del problema

Una organización mantiene un sistema que sostiene gran parte de su operación administrativa y operativa, integrando sus diferentes áreas. La información se almacena en dos motores de base de datos, uno relacional y otro documental. El segundo, al no ser relacional, no implementa llaves foráneas como restricción del propio motor y, al tratarse de motores distintos, tampoco hay un mecanismo de integridad referencial que los conecte entre sí, por lo que la relación entre los datos depende de una convención y no de una restricción verificada por el sistema.

El sistema, desarrollado con Laravel 8 y PHP 8.3 y mantenido por cinco personas, nació hace dos años orientado principalmente a procesos operativos, con funciones como la generación de resguardos de unidad y bitácoras de servicio. Con el tiempo escaló hacia el área administrativa e incorporó de forma progresiva nuevas funcionalidades hasta convertirse en el sistema principal de la organización, sin que su arquitectura se revisara al mismo ritmo que su alcance. Hoy sostiene operaciones críticas, mientras que su proceso de despliegue continúa siendo completamente manual.

Cualquiera de los cinco integrantes del equipo puede realizar un despliegue, y para hacerlo es necesario acceder mediante SSH al servidor donde se aloja la aplicación, actualizar el código mediante operaciones de Git sobre el repositorio y ejecutar manualmente las migraciones pendientes, sin mecanismos de verificación automática. El historial del repositorio refleja un volumen considerable de cambios: desde su inicio se han fusionado 563 solicitudes de cambio en la rama principal, sobre aproximadamente 8,000 *commits*. Estas integraciones pueden incluir cambios que deben aplicarse sobre los motores de base de datos: modificación de esquemas en el motor relacional y, en el motor no relacional, que no tiene esquema fijo, operaciones equivalentes como la creación de índices; en ambos casos deben identificarse y ejecutarse manualmente y en el orden correspondiente.

El ritmo de integración de cambios a la rama principal no equivale a la frecuencia real de despliegue, ya que los pases a producción se realizan de forma manual y a menudo agrupan varios cambios. A pesar de no contar con una telemetría exacta de los despliegues manuales, el análisis histórico del repositorio durante los últimos seis meses permite establecer una línea base cuantitativa: se han registrado 338 integraciones a la rama principal, lo que representa una frecuencia máxima teórica de aproximadamente 13 integraciones por semana.

El tiempo mediano de entrega de los cambios (*lead time for changes*), medido desde el primer *commit* hasta su fusión en la rama principal, es de aproximadamente 55 minutos, aunque la existencia de ramas de larga duración eleva el promedio general a 2,0 días. Por otro lado, la tasa de fallos (*change failure rate*), estimada mediante la identificación de ramas de corrección, indica que el 21,9 % de las integraciones (74 de 338) requirieron una intervención correctiva posterior. El proceso manual actual, al depender de pasos ejecutados a mano y carecer de entornos de prueba automatizados, propicia que un porcentaje considerable de los cambios llegue a la rama principal requiriendo ajustes posteriores.

El proceso manual, al depender de pasos ejecutados a mano, presenta un riesgo propio de los despliegues que agrupan varios cambios: omitir alguna migración o *seeder* de los cambios acumulados. Hasta ahora no ha ocurrido, y los pocos olvidos de ejecución que se han presentado se corrigieron con rapidez y sin consecuencias relevantes.

1.1. Antecedentes

El primer elemento a considerar al intervenir un proceso de entrega de software es establecer un mecanismo sistemático y validado para medir su desempeño antes y después de la automatización. Forsgren, Humble y Kim [1] sintetizaron años de investigación empírica en miles de organizaciones y aislaron cuatro métricas clave, conocidas en la literatura como métricas DORA: frecuencia de despliegue (*deployment frequency*), tiempo de entrega de los cambios (*lead time for changes*), tiempo de restauración del servicio (*time to restore service*) y tasa de fallos por cambio (*change failure rate*). El hallazgo central demuestra que las organizaciones de alto desempeño no sacrifican estabilidad por velocidad, sino que consiguen ambas de forma simultánea. Para este trabajo, estas métricas proporcionan un vocabulario y una definición operacional ya validados, y ofrecen un marco científico para cuantificar la mejora respecto a la línea base (*baseline*) de los despliegues manuales.

Sobre la adopción de la integración y entrega continuas (CI/CD), existe un consenso sólido respecto a las principales barreras de implementación. Revisiones sistemáticas de literatura empírica desarrolladas por Laukkanen et al. [2] y Proulx et al. [3] coinciden en que los factores humanos y organizacionales superan holgadamente a los problemas técnicos en la generación de fallas durante la liberación de código. Estos hallazgos concuerdan con lo que se observa en el sistema de estudio, cuyo principal riesgo de despliegue (omitir una migración cuando se agrupan varios cambios) depende de que una persona ejecute cada paso a mano. La literatura señala que la automatización mediante un *pipeline* establece un mecanismo restrictivo de construcción que puede reducir la posibilidad de errores humanos por omisión al hacer explícitos y verificables los pasos del despliegue.

Al intentar aplicar estos principios a sistemas *legacy*, la literatura técnica se inclina hacia una refactorización profunda o la descomposición en microservicios. Sin embargo, diferentes estudios recientes en la ingeniería de software empresarial advierten sobre los elevados riesgos y costos que puede ocasionar esta estrategia, especialmente en infraestructura restringida y autoalojada. La práctica documentada respalda la modernización incremental [4], un enfoque que prioriza la contenedorización de la aplicación tal como está, aislando sus dependencias de tiempo de ejecución (*runtime*) sin reescribir la lógica del negocio. Este es un paso crítico en contextos que operan sobre infraestructura propia y carecen de orquestación en la nube pública.

Un desafío arquitectónico particular del sistema en cuestión es su dependencia de un esquema de base de datos dual y heterogéneo (un motor relacional y uno documental simultáneos), desprovisto de un mecanismo nativo de integridad referencial. Estudios empíricos sobre entrega continua, como el de Shahin et al. [5], identifican la sincronización y la evolución de esquemas de bases de datos como uno de los cuellos de botella técnicos más severos al automatizar despliegues. Dado que no existe una gestión de transacciones distribuidas entre ambos motores, cualquier estrategia de automatización debe tratar ambas bases como una unidad lógica al ejecutar migraciones o *backfills* de datos. Esto impone una restricción de diseño importante para el *pipeline* propuesto.

Por otra parte, el obstáculo más significativo frente a la literatura tradicional de CI/CD es la validación del código. La inmensa mayoría de los trabajos académicos asumen la preexistencia de una *suite* de pruebas automatizadas como el *quality gate* (compuerta de calidad) principal. Cuando un sistema *legacy* carece de pruebas unitarias o de integración, forzar la entrega continua presenta un riesgo crítico. Para cerrar esta brecha en escenarios con datos altamente acoplados, se recomienda desplegar los cambios en entornos de prueba bajo demanda (*on-demand preview environments*),

donde una persona puede validar el cambio de forma manual en condiciones parecidas a las de producción antes de integrarlo.

Operar con servidores propios (*on-premise*) añade una capa final de complejidad debido a restricciones presupuestales y a la falta de registros de contenedores (*container registries*) preexistentes. Un precedente altamente relevante en este rubro es el trabajo de Morales Agredo [6], quien diseñó en la Pontificia Universidad Javeriana Cali una propuesta de CI/CD orientada a aplicaciones *legacy* universitarias. Su investigación demuestra que la transición hacia flujos automatizados con plataformas autoalojadas (*self-hosted*) es viable e imperativa en organizaciones con infraestructura limitada, sin depender de proveedores externos.

La Tabla 1 resume las características de los trabajos revisados frente a las condiciones de este proyecto.

Trabajo	**CI/CD**	***Legacy***	***On-premise***	**Sin pruebas automatizadas**
Forsgren et al. [1]	Mide su efecto con las métricas DORA	No es el foco	No distingue entre nube y servidores propios	No, considera las pruebas una capacidad clave
Laukkanen et al. [2]	Problemas al adoptar la entrega continua	Señala arquitecturas poco modulares como obstáculo	No es el foco	No, las asume y reporta pruebas lentas o inestables
Proulx et al. [3]	Problemas y soluciones del despliegue continuo	No es el foco	No es el foco	No; las asume existentes
Kosińska y Zielinski [4]	Entrega continua sobre contenedores	Modernización incremental sin reescribir la lógica	No, orientado a entornos nativos de la nube	No es el foco
Shahin et al. [5]	Enfoques, herramientas y desafíos, incluida la evolución de esquemas de BD	No es el foco	No es el foco	No, las asume como compuerta de calidad
Morales Agredo [6]	Propuesta para aplicaciones universitarias	Aplicaciones *legacy* universitarias	Sí, plataformas autoalojadas	Por verificar
Este trabajo	*Pipeline* en GitLab con entornos de prueba bajo demanda y aprobación manual	Monolito Laravel con MySQL y MongoDB, sin reescritura	Sí, servidores propios con GitLab autoalojado	Sí, compuertas de calidad y validación manual en instancias de prueba

Tabla 1: Características de los trabajos revisados frente a este trabajo.

Al cruzar los ejes anteriores, emerge con claridad la brecha técnica y operativa que motiva este trabajo. La literatura revisada documenta sólidamente cómo evaluar el desempeño, los riesgos del error humano, la viabilidad de la contenedorización incremental y la modernización de sistemas *legacy* con plataformas autoalojadas. Ninguno de estos trabajos, sin embargo, aborda de manera conjunta la automatización de la entrega de un sistema monolítico con un esquema de datos heterogéneo, operado sobre infraestructura *on-premise* sin acceso a nube pública, y que deba garantizar la

seguridad del despliegue en ausencia total de una *suite* de pruebas automatizadas. Es precisamente en la resolución simultánea de estas restricciones operativas donde se inscribe la contribución de este proyecto.

1.2. Definición del problema

Situación actual. El despliegue del sistema a producción es completamente manual y no incluye ninguna verificación automática: cada paso, incluidas las migraciones de las dos bases de datos, lo ejecuta a mano quien despliega. Los cambios se revisan entre el equipo antes de fusionarse, pero no hay pruebas automatizadas ni un entorno que reproduzca las condiciones de producción, y no se registra cuánto tardan los cambios en llegar a producción ni cuántos despliegues requieren una corrección.

Consecuencias. Cada despliegue depende de que quien lo realice recuerde y ejecute en orden todos los pasos, una tarea frecuente que se reparte entre distintas personas. Hasta ahora esto ha causado pocos problemas, pero cuando se agrupan varios cambios basta un descuido para omitir una migración. Además, la revisión entre pares es la única barrera antes de producción: lo que no detecte llega directamente a un sistema del que dependen operaciones críticas, y sin registros no hay forma de saber con qué frecuencia ocurre.

Problema central. Dificultad para desplegar y probar los cambios del sistema de forma confiable y repetible: el despliegue depende de pasos manuales que realizan distintas personas, los cambios no pueden probarse en condiciones parecidas a las de producción antes de llegar a ella y los datos están repartidos en dos bases de datos que deben actualizarse de manera coordinada.

1.3. Justificación

La automatización del ciclo de entrega de software no busca únicamente acelerar el desarrollo de la aplicación, sino reducir los riesgos de operar un sistema crítico. En el sistema de estudio, la seguridad de cada despliegue descansa en la memoria de quien lo ejecuta y en la revisión entre pares, porque no existen pruebas automatizadas ni un entorno de validación al que pueda acceder un usuario final. Mientras sea así, cualquier defecto que escape a la revisión o cualquier paso omitido termina afectando a quienes usan el sistema.

Un proceso automatizado con entornos de prueba bajo demanda atiende esta situación de tres maneras. Primero, sustituye los pasos manuales por un *pipeline* que los ejecuta siempre igual, sin importar quién despliegue. Segundo, cuando se requiere, permite validar un cambio en una instancia de prueba antes de fusionarlo, mientras que las compuertas de calidad del *pipeline* detectan de forma automática algunos defectos desde que se sube el cambio. Ninguno de estos controles garantiza la ausencia de errores, pero aumentan la probabilidad de detectarlos antes de llegar a producción. Tercero, deja registro de cada despliegue, algo que hoy no existe y que permite evaluar el proceso con datos.

Esto beneficia al equipo técnico, al reducir su dependencia de procedimientos manuales y favorecer un proceso estandarizado, repetible y con trazabilidad. También beneficia a la organización,

al disminuir el riesgo de que un error afecte la operación de un sistema que sustenta su gestión administrativa. Además, el proyecto deja documentada una *estrategia reproducible de modernización progresiva* para sistemas *legacy* sin pruebas automatizadas y sobre infraestructura autoalojada, sirviendo de referencia metodológica y operativa a equipos con condiciones similares.

1.4. Objetivo general

Diseñar e implementar un proceso de integración y entrega continua para el sistema, basado en contenedores y en entornos de prueba bajo demanda sobre infraestructura autoalojada, que reduzca los pasos manuales del despliegue y permita validar los cambios antes de su llegada a producción, y evaluarlo frente al proceso manual actual.

1.5. Objetivos específicos

Para alcanzar el objetivo general propuesto, se plantean los siguientes objetivos específicos:

- Empaquetar la aplicación en imágenes de contenedor versionadas, sin modificar su lógica de negocio, para que se ejecute de la misma forma en desarrollo, pruebas y producción.
- Implementar el despliegue de prueba bajo demanda: que cualquier integrante del equipo pueda elegir desde GitLab una rama y una instancia de prueba, y desplegar ahí la aplicación con sus dos bases de datos (MySQL y MongoDB), con opciones para aplicar migraciones y cargar datos de prueba generados mediante *seeders*.
- Configurar en GitLab, sobre infraestructura autoalojada, un *pipeline* que en cada cambio que se sube al repositorio construya la imagen y lo verifique mediante compuertas de calidad, ante la ausencia de una *suite* de pruebas automatizadas.
- Automatizar el despliegue a producción a partir de una rama de liberación (*release*), con aprobación manual, ejecutando en orden las migraciones de ambas bases de datos y verificando al final que la aplicación responde.
- Evaluar el proceso propuesto frente a la línea base del proceso manual actual, utilizando las *métricas DORA* adaptadas al contexto del sistema (frecuencia de despliegue, *lead time for changes* y tasa de fallos), sumado a la reducción de pasos manuales del despliegue.

1.6. Hipótesis

[Opcional según el tipo de trabajo.]

1.7. Alcances y limitaciones

Alcances. El proyecto se limita a un sistema, desarrollado con Laravel, MySQL y MongoDB, y cubre: la contenedorización de la aplicación sin modificar su lógica; un *pipeline* en GitLab autoalojado con compuertas de calidad; el despliegue de prueba bajo demanda en instancias de prueba; el despliegue a producción desde una rama de liberación con aprobación manual, con sus migraciones; y la comparación con el proceso manual actual.

No incluye. La reescritura del sistema ni su división en microservicios. Tampoco incluye alcanzar una cobertura total o significativa de pruebas automatizadas para la lógica de negocio existente, aunque sí contempla dejar configurado e inicializado el entorno (*framework* de pruebas) dentro del *pipeline* para su adopción futura.

Limitaciones. Al tratarse de un solo sistema, los resultados no pueden generalizarse sin más a otros. Las compuertas de calidad no sustituyen a una *suite* de pruebas, y en la instancia de prueba los cambios se siguen validando a mano. Las instancias de prueba son limitadas en número y cada una prueba una rama a la vez. Los datos de prueba no reproducen todos los casos de uso reales. La línea base del proceso manual se reconstruye a partir de registros parciales, porque antes no se medía.

2. Marco teórico

Esta sección se encarga de definir y explicar los conceptos y prácticas de ingeniería de software a los que se hará referencia a lo largo de este documento. El objetivo es proporcionar al lector los conocimientos teóricos necesarios para comprender el producto y la metodología propuesta. Dado que este proyecto aborda la modernización del proceso de liberación de un sistema *legacy*, se explicarán a detalle términos como la integración y entrega continuas (CI/CD), las métricas de desempeño del desarrollo de software (DORA), la contenedorización de aplicaciones, la función de las compuertas de calidad y los marcos de trabajo ágiles.

2.1. Integración y Entrega Continuas (CI/CD)

En el desarrollo de software moderno, la velocidad y la confiabilidad al momento de liberar nuevas características a los usuarios son fundamentales. La Integración Continua (CI, por sus siglas en inglés) es una práctica de desarrollo en la cual los miembros de un equipo integran su código fuente con frecuencia en un repositorio central, donde cada integración es verificada automáticamente por un sistema que construye el código y lo somete a pruebas [2]. El objetivo principal de la CI es detectar errores de integración lo más rápido posible, brindando retroalimentación inmediata a los desarrolladores antes de que los defectos lleguen a entornos críticos.

Por su parte, la Entrega Continua (CD) es la evolución natural de la CI. Consiste en asegurar que el software pueda ser liberado de forma confiable en cualquier momento, manteniendo el código en un estado siempre desplegable. Es importante distinguir entre Entrega Continua y Despliegue Continuo: mientras que el Despliegue Continuo automatiza la liberación a producción sin intervención

humana, la Entrega Continua requiere una aprobación manual final antes del pase a producción, lo que la hace idónea para sistemas críticos [3].

Al intentar aplicar estas prácticas a sistemas *legacy* (sistemas heredados), los equipos se enfrentan a arquitecturas poco modulares y acoplamientos fuertes de bases de datos que dificultan la adopción de flujos CI/CD automatizados [2, 5]. En estos contextos organizacionales, la literatura señala que la implementación de un *pipeline* de CI/CD actúa como un mecanismo restrictivo y documentado, reduciendo drásticamente la posibilidad de errores humanos por omisión que suelen ocurrir durante los despliegues manuales [1].

2.2. Métricas DORA para el Desempeño del Desarrollo

Al intervenir un proceso de entrega de software, es indispensable contar con un mecanismo validado para medir el desempeño antes y después de la automatización. A través de investigaciones empíricas en la industria, se ha demostrado que es posible medir la efectividad del ciclo de vida del software sin sacrificar la estabilidad por la velocidad. Forsgren, Humble y Kim [1] identificaron cuatro métricas clave, comúnmente denominadas métricas DORA (por el grupo de investigación *DevOps Research and Assessment*), que permiten evaluar el rendimiento de la entrega de software:

- **Frecuencia de despliegue (*Deployment frequency*):** Mide qué tan a menudo la organización despliega código a producción. Una mayor frecuencia indica integraciones más pequeñas y de menor riesgo.
- **Tiempo de entrega de los cambios (*Lead time for changes*):** Mide el tiempo que transcurre desde que un código es confirmado (*commit*) en el repositorio hasta que se ejecuta exitosamente en producción.
- **Tasa de fallos por cambio (*Change failure rate*):** Representa el porcentaje de despliegues a producción que provocan una falla en el servicio y requieren una intervención inmediata (como un *rollback* o un parche de emergencia).
- **Tiempo de restauración del servicio (*Time to restore service*):** Mide el tiempo que tarda la organización en recuperarse de una falla en producción.

El uso de estas métricas proporciona una definición operacional y un vocabulario científico validado para cuantificar la mejora de los equipos técnicos frente a sus líneas base históricas, demostrando el valor empírico de adoptar prácticas como CI/CD [1].

2.3. Contenedorización e Infraestructura Inmutable

El despliegue de aplicaciones heredadas (*legacy*) en entornos de servidores propios (*on-premise*) suele enfrentarse al problema de la inconsistencia de los entornos. Lo que funciona en la máquina de desarrollo a menudo falla en producción debido a diferencias sutiles en las versiones de bibliotecas, sistemas operativos o variables de entorno. Para mitigar este problema, la industria ha adoptado la contenedorización como un estándar de facto.

La contenedorización es una tecnología de virtualización a nivel del sistema operativo que permite empaquetar una aplicación y todas sus dependencias (bibliotecas, archivos binarios y configuración) en un único artefacto ejecutable denominado ïmagen"[7]. A diferencia de las máquinas virtuales tradicionales, los contenedores comparten el núcleo del sistema operativo del servidor anfitrión, lo que los hace extremadamente ligeros, con tiempos de arranque en el orden de los milisegundos.

Al emplear contenedores, se habilita el patrón de diseño arquitectónico conocido como ïnfraestructura inmutable". Este principio dicta que, una vez que una imagen de contenedor es construida por el *pipeline* de integración continua, esta no se modifica ni se parchea jamás. Si se requiere una actualización o un cambio en el código, se debe construir una imagen completamente nueva y sustituir a la anterior [7]. En el contexto de sistemas *legacy*, este enfoque es invaluable: permite encapsular el entorno de ejecución (*runtime*) sin necesidad de reescribir la lógica del negocio, garantizando que el código probado en un servidor temporal bajo demanda se ejecutará de forma idéntica en producción.

2.4. Compuertas de Calidad (*Quality Gates*) en el Desarrollo

La automatización del despliegue (CD) introduce la necesidad crítica de asegurar que el código integrado al repositorio cumpla con los estándares de la organización antes de llegar al servidor. Este proceso de validación automática se gestiona a través de las compuertas de calidad o *quality gates*.

Una compuerta de calidad es un hito dentro del *pipeline* que detiene la integración de manera automatizada si el código no cumple con criterios predefinidos [8]. Históricamente, la compuerta de calidad principal ha sido la ejecución de *suites* de pruebas dinámicas (pruebas unitarias o de integración). Sin embargo, en sistemas donde la creación retrospectiva de pruebas es prohibitiva por costos de tiempo o acoplamiento de bases de datos, las organizaciones adoptan el enfoque *DevSecOps* mediante análisis estáticos y de seguridad [8].

Las compuertas de análisis estático (SAST - *Static Application Security Testing*) evalúan el código fuente sin ejecutarlo, detectando problemas de sintaxis, violaciones de arquitectura, o estándares de estilo inconsistentes. Adicionalmente, el escaneo de vulnerabilidades sobre las imágenes de contenedores permite detectar dependencias obsoletas o fallos de seguridad documentados (CVEs) antes de autorizar el despliegue. Según la literatura reciente, transferir la carga de revisión desde el desarrollador hacia estas compuertas automatizadas reduce el error humano y garantiza un estándar de calidad mínimo indispensable en la integración de ramas [8].

2.5. Metodologías Ágiles aplicadas a la Modernización

La modernización de infraestructura y la implementación de ciclos automatizados en sistemas ya existentes implican altos niveles de incertidumbre técnica. A diferencia del desarrollo de un sistema desde cero, la intervención de un sistema *legacy* suele revelar dependencias ocultas, deuda técnica y requerimientos operativos no documentados a medida que avanza el trabajo [9].

Por esta razón, abordar un proyecto de automatización *DevOps* mediante metodologías tradicionales (como el modelo en cascada o ciclo de vida lineal) eleva el riesgo de fracaso por descubrimiento tardío de problemas. Las metodologías ágiles, y en particular marcos de trabajo iterativos como

Scrum, proveen una estructura organizativa que fomenta la adaptación continua [9]. Al dividir el trabajo técnico en iteraciones cortas (*sprints*), el equipo puede entregar incrementos funcionales (por ejemplo, automatizar primero la contenedorización y luego las bases de datos), medir el impacto, y ajustar el curso de acción ante los retos técnicos que presente la infraestructura *on-premise*.

3. Producto propuesto

El producto de este proyecto es un proceso de integración y entrega continua para el sistema, diseñado para operar sobre la infraestructura autoalojada de la organización. Todos sus elementos se describen en archivos versionados dentro del repositorio (infraestructura como código), por lo que el proceso es repetible y queda registrado. Se compone de:

- Las imágenes de contenedor de la aplicación y su configuración.
- Un *pipeline* en GitLab que, en cada cambio que se sube al repositorio, construye la imagen y ejecuta las compuertas de calidad.
- El despliegue de prueba bajo demanda: desde GitLab se elige una rama y una instancia de prueba, y se despliega ahí la aplicación con sus dos bases de datos, con la opción de aplicar migraciones y cargar datos de prueba.
- El despliegue a producción desde una rama de liberación, con aprobación manual, que ejecuta en orden las migraciones de ambas bases de datos y verifica que la aplicación responda.
- La documentación de uso y el informe de evaluación frente al proceso manual actual.

El proceso funciona de la siguiente manera (Figura 1). Cuando un desarrollador sube una rama, GitLab inicia un *pipeline* de revisión que construye la imagen y ejecuta las compuertas de calidad. Si el desarrollador lo requiere, puede desplegar su rama en una instancia de prueba para que el cambio se valide de forma manual, de preferencia por quien lo usará. Cuando el cambio pasa las verificaciones y las aprobaciones del equipo, se fusiona con la rama principal. Para llevarlo a producción se crea una rama de liberación a partir de la principal; sobre ella se ejecuta el *pipeline* de despliegue, que vuelve a ejecutar las verificaciones y, tras una aprobación manual, despliega la aplicación en producción.

Las características principales del proceso son:

- **Imágenes versionadas.** Las imágenes de todos los ambientes se construyen a partir de la misma definición y de la misma imagen base, lo que reduce las diferencias de configuración entre ambientes.
- **Infraestructura autoalojada.** GitLab, su registro de contenedores y los servidores donde se despliega son propios y se administran internamente.

- **Despliegue de prueba bajo demanda.** Una instancia de prueba es un servidor interno donde se despliega la aplicación para validar un cambio. Cualquier integrante puede desplegar una rama en una de ellas, con datos de prueba generados mediante *seeders* que respetan las relaciones entre MySQL y MongoDB. Las instancias son persistentes y cada una aloja una rama a la vez. La validación funcional sigue siendo manual: la instancia la hace posible y ordenada, pero no la sustituye.
- **Despliegue coordinado de las dos bases de datos.** Las migraciones de ambos motores se ejecutan en un orden definido y antes de exponer el código nuevo. Como los motores no comparten transacciones, el proceso asegura el orden y la verificación, no la atomicidad.
- **Controles en producción.** El despliegue requiere la aprobación manual de un rol autorizado y no permite borrar la base de datos.

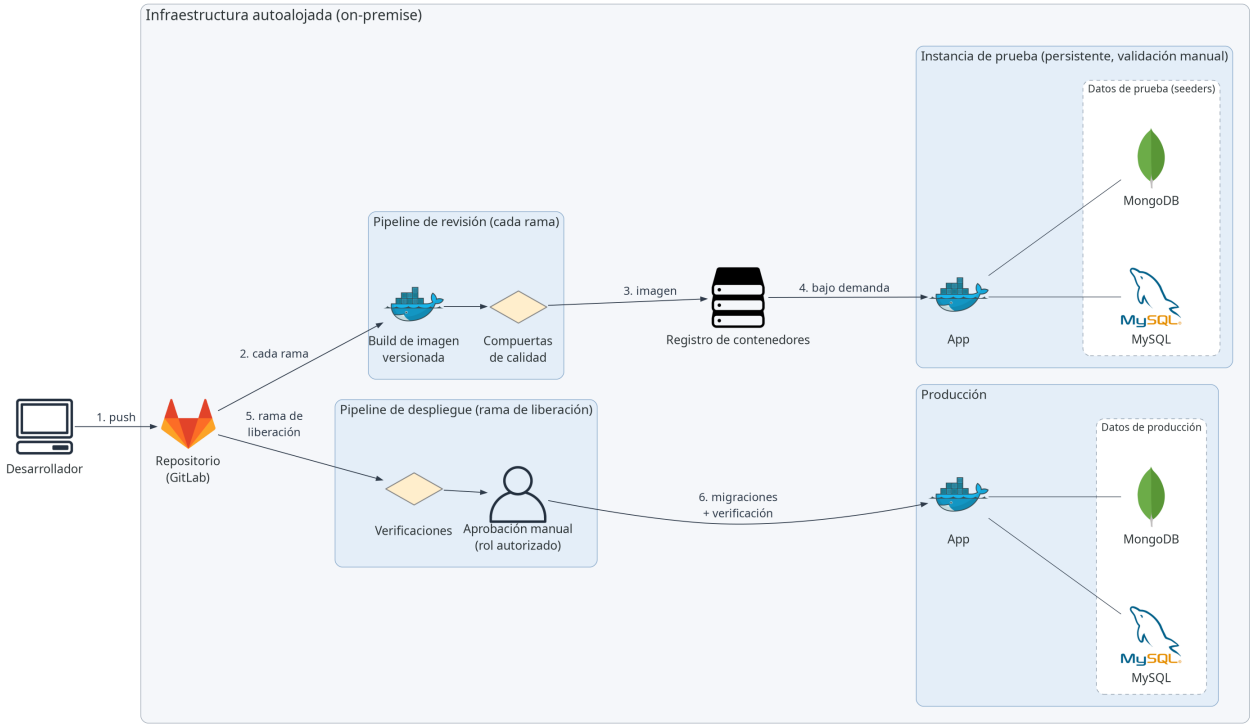


Figura 1: Arquitectura propuesta para la entrega continua.

3.1. Métricas de aceptación

Para determinar el éxito y la viabilidad del producto propuesto tras su implementación, se establecen los siguientes criterios operativos, evaluados frente a la línea base histórica:

- **Reducción de pasos manuales:** El proceso de despliegue a producción debe reducirse a una única aprobación manual (clic) en la interfaz de GitLab, delegando la construcción de imágenes, ejecución de migraciones y verificación al *pipeline* automatizado.

- **Tiempo máximo de despliegue:** La ejecución del *pipeline* de liberación a producción deberá completarse de forma predecible y documentada, estandarizando el tiempo que toma colocar el código en el servidor.
- **Impacto en el *lead time*:** La adopción del proceso automatizado y las compuertas de calidad no deberá generar una degradación significativa en el *lead time* mediano actual de 55 minutos, demostrando que la automatización no es un cuello de botella.
- **Porcentaje de despliegues exitosos:** Se busca disminuir la tasa histórica de fallos (estimada actualmente en un máximo del 21.9 %) mediante la detección temprana de defectos en el *pipeline* de revisión y la validación en los entornos de prueba bajo demanda.

4. Metodología

Para llevar a cabo el desarrollo de este proyecto se optó por un enfoque basado en una metodología ágil adaptada del marco de trabajo *Scrum*, lo que permitirá una respuesta flexible ante los retos técnicos inherentes a la intervención de sistemas *legacy*. El trabajo se estructurará en iteraciones cortas o *sprints*, cuyas tareas, incidencias y avances serán gestionados a través de tableros para el control del flujo de trabajo. De esta forma, cada incremento iterativo entregará una parte funcional y medible de la automatización del *pipeline*. Técnicamente, la ejecución se estructurará en las siguientes seis fases progresivas:

- **Fase 1. Diagnóstico y línea base**: Se realizará un levantamiento detallado del proceso de despliegue manual actual. Se extraerán las métricas históricas del repositorio en GitLab para consolidar la línea base cuantitativa, estableciendo la frecuencia de despliegue, el *lead time* y la tasa de fallos actual.
- **Fase 2. Contenedorización**: Se empaquetará (*dockerización*) la aplicación web. Para mantener el rendimiento del servidor CI, se diseñará una estrategia de construcción basada en imágenes de múltiples capas (separando la compilación de extensiones base del código de la aplicación). Se ejecutará una validación funcional para garantizar que el aislamiento del *runtime* no altere la lógica de negocio ni la conexión entre los motores.
- **Fase 3. Pipeline CI y Compuertas de calidad**: Se configurará en GitLab CI el flujo automático de construcción de imágenes. Ante la ausencia histórica de una *suite* de pruebas automatizadas, el *pipeline* se enfocará inicialmente en implementar compuertas de calidad basadas en análisis estático y seguridad. Esto contemplará la validación de la sintaxis del código, la revisión estandarizada de estilo y el escaneo de vulnerabilidades en los contenedores, utilizando herramientas compatibles con el ecosistema actual del proyecto. Adicionalmente, se integrará e inicializará la base del *framework* de pruebas dentro del contenedor de integración, sentando la infraestructura necesaria para que el equipo comience una adopción progresiva de pruebas en futuros desarrollos.
- **Fase 4. Entornos de prueba bajo demanda**: Se habilitarán instancias de prueba persistentes en la infraestructura autoalojada, gestionadas mediante herramientas como SSH y Docker

Compose, en las que se desplegará una rama a la vez. Desde el contenedor de la aplicación desplegada se automatizará la ejecución segura de migraciones y *seeders* hacia las bases de datos correspondientes al entorno de pruebas, permitiendo validaciones manuales antes de la integración.

- **Fase 5. Pipeline CD (Entrega continua)**: Se automatizará la liberación a producción mediante un flujo protegido por aprobación manual en la plataforma. El despliegue automatizado garantizará la ejecución ordenada de las migraciones necesarias en MySQL y MongoDB correspondientes a la nueva versión de la aplicación, eliminando la necesidad de intervención manual por SSH.
- **Fase 6. Evaluación de Resultados**: Se comparará empíricamente el nuevo proceso automatizado frente a la línea base histórica obtenida en la Fase 1. Se determinará el cumplimiento de las métricas de aceptación y se documentará el impacto final en el *lead time* y la tasa de errores.

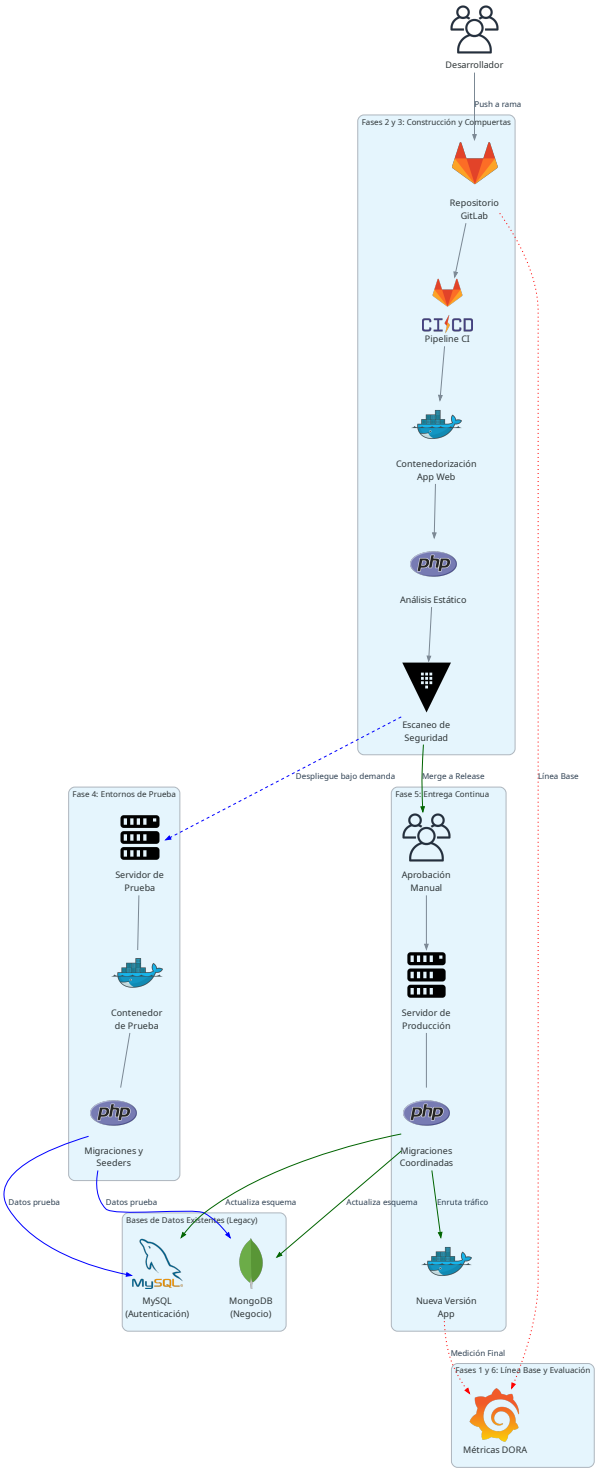


Figura 2: Flujo metodológico del proceso CI/CD propuesto.

5. Programa de actividades

El desarrollo de este proyecto está planificado para ejecutarse en un periodo de seis meses. Las actividades se han estructurado tomando como base la metodología iterativa propuesta, asignando tiempos específicos para el desarrollo técnico de las seis fases, así como periodos dedicados a la redacción del informe final, la revisión por parte de los asesores y un margen contemplado para imponderables o imprevistos técnicos inherentes a la intervención de sistemas *legacy*.

A continuación, en la Tabla 2, se presenta el cronograma mediante un diagrama de Gantt que detalla la distribución de la carga de trabajo mensual.

Actividad / Fase	**Mes 1**	**Mes 2**	**Mes 3**	**Mes 4**	**Mes 5**	**Mes 6**
1. Fase 1: Diagnóstico y extracción de métricas base en GitLab						
2. Fase 2: Contenedorización de la aplicación y estrategia de imágenes						
3. Fase 3: Configuración del Pipeline CI y compuertas de calidad						
4. Fase 4: Despliegue de entornos de prueba bajo demanda y *seeders*						
5. Fase 5: Automatización del Pipeline CD (Entrega a producción)						
6. Fase 6: Evaluación de resultados y comparación de métricas DORA						
7. Redacción de capítulos del informe final de titulación						
8. Revisiones con asesor, correcciones e imponderables						

Tabla 2: Cronograma de actividades (Diagrama de Gantt).

Como se observa en el cronograma, la redacción del documento no se posterga hasta el final del proyecto, sino que se ejecuta de forma paralela al desarrollo técnico a partir del segundo mes, garantizando que los hallazgos y decisiones de infraestructura se documenten con inmediatez.

6. Resultados esperados

Al finalizar el desarrollo e implementación de este proyecto, se espera obtener los siguientes resultados, divididos en entregables técnicos, impacto operativo y valor estratégico para la organización:

1. Entregables técnicos funcionales

- Un *pipeline* de integración y entrega continuas (CI/CD) operativo en la plataforma GitLab autoalojada de la organización.
- La aplicación web *legacy* empaquetada en imágenes de contenedor (Docker) versionadas y construidas por capas, estandarizando su ejecución sin alterar la lógica de negocio actual.
- El despliegue bajo demanda de una rama en instancias de prueba persistentes, con la opción de aplicar migraciones y cargar datos de prueba (*seeders*) en MySQL y MongoDB, para validar de forma manual los cambios funcionales antes de su fusión.

2. Impacto operativo y evaluación empírica

- La sustitución de los pasos manuales del despliegue por SSH por un flujo automatizado y trazable, en el que la intervención humana se reduce a una aprobación manual (un clic) desde la interfaz de GitLab.
- La reducción del riesgo de omitir migraciones al agrupar cambios, ya que el *pipeline* aplicará las migraciones de MySQL y MongoDB en un orden definido y verificará el resultado en cada liberación. Al no compartir transacciones ambos motores, se asegura el orden y la verificación, no la atomicidad.
- Un informe de evaluación que contraste el desempeño del nuevo flujo contra la línea base histórica del proceso manual (Apéndice **??**), utilizando las métricas DORA adaptadas (frecuencia de despliegue, *lead time* y tasa de fallos) y la reducción de pasos manuales. Por tratarse de un solo sistema y de una línea base reconstruida a partir de registros parciales, los resultados serán específicos de este contexto.

3. Valor estratégico e infraestructura futura

- Compuertas de calidad automatizadas, basadas en análisis estático del código y escaneo de vulnerabilidades de las imágenes, que detengan la integración de los cambios que no cumplan los criterios definidos. Estas compuertas no sustituyen a una *suite* de pruebas, pero aumentan la probabilidad de detectar defectos antes de llegar a producción.
- La integración e inicialización de un *framework* de pruebas dentro del *pipeline*, dejando el entorno preparado para que el equipo inicie una adopción progresiva de pruebas automatizadas.
- La documentación del proceso como referencia para la organización y para equipos en condiciones similares, mostrando una estrategia de modernización progresiva de la entrega de un sistema *legacy* sobre infraestructura propia, sin reescribir su lógica de negocio.

Referencias

[1] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations*. Portland, OR: IT Revolution Press, 2018.

[2] E. Laukkanen, J. Itkonen, and C. Lassenius, "Problems, causes and solutions when adopting continuous delivery—a systematic literature review," *Information and Software Technology*, vol. 82, pp. 55–79, 2017.

[3] A. Proulx, F. Raymond, B. Roy, and F. Petrillo, "Problems and solutions of continuous deployment: A systematic review," 2018. Preprint, no arbitrado.

[4] J. Kosińska and K. Zielinski, "Cloud-native enterprise engineering: design, automation, and continuous delivery," *Software: Practice and Experience*, vol. 50, no. 8, pp. 1415–1443, 2020.

[5] M. Shahin, M. A. Babar, and L. Zhu, "Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices," *IEEE Access*, vol. 5, pp. 3909–3943, 2017.

[6] A. Morales Agredo, "Propuesta de implementación de Integración Continua (CI) y Despliegue Continuo (CD) en aplicaciones *legacy* de la Pontificia Universidad Javeriana Cali," trabajo de grado, Pontificia Universidad Javeriana, Cali, Colombia, 2021.

[7] D. Taibi, D. Lenko, and C. Pahl, "Migrating legacy systems to cloud-native architectures using containerization and immutable infrastructure," *Journal of Systems and Software*, vol. 192, p. 111400, 2022.

[8] A. P. Rajesh, S. Kumar, and M. Singh, "Integrating security into ci/cd pipelines: A devsecops approach and the role of quality gates," *IEEE Transactions on Software Engineering*, vol. 49, no. 4, pp. 1542–1559, 2023.

[9] M. C. Vargas, R. Silva, and H. Souza, "Agile practices in the maintenance and modernization of legacy software systems," in *Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pp. 234–245, IEEE, 2021.

[10] S. Berner and otros, "Problemas que afectan a la calidad de software en entrega continua," in *XXIV Congreso Argentino de Ciencias de la Computación (CACIC)*, (Tandil, Argentina), 2018.

A. Apéndice

[Material secundario o demasiado extenso para el cuerpo: código fuente completo, derivaciones, tablas de resultados completos, convenios, etc. Va después de la literatura citada y no lleva introducción ni conclusión.]